

Submission Worksheet

Submission Data

Course: IT114-450-M2025

Assignment: IT114 Milestone 1

Student: Alex P. (ap2869)

Status: Submitted | **Worksheet Progress:** 100%

Potential Grade: 10.00/10.00 (100.00%)

Received Grade: 0.00/10.00 (0.00%)

Started: 7/5/2025 8:31:10 PM

Updated: 7/7/2025 3:31:20 PM

Grading Link: <https://learn.ethereallab.app/assignment/v3/IT114-450-M2025/it114-milestone-1/grading/ap2869>

View Link: <https://learn.ethereallab.app/assignment/v3/IT114-450-M2025/it114-milestone-1/view/ap2869>

Instructions

- Overview Link: <https://youtu.be/9dZPFwi76ak>

1. Refer to Milestone1 of any of these docs:

2. [Rock Paper Scissors](#)
3. [Basic Battleship](#)
4. [Hangman / Word guess](#)
5. [Trivia](#)
6. [Go Fish](#)
7. [Pictionary / Drawing](#)

2. Ensure you read all instructions and objectives before starting.

3. Ensure you've gone through each lesson related to this Milestone

4. Switch to the Milestone1 branch

1. `git checkout Milestone1` (ensure proper starting branch)
2. `git pull origin Milestone1` (ensure history is up to date)

5. Copy Part5 and rename the copy as Project (this new folder should be in the root of your repo)

6. Organize the files into their respective packages Client, Common, Server, Exceptions

1. Hint: If it's open, you can refer to the Milestone 2 Prep lesson

7. Fill out the below worksheet

1. Ensure there's a comment with your UCID, date, and brief summary of the snippet in each screenshot
2. Since this Milestone was majorly done via lessons, the required comments should be placed in areas of analysis of the requirements in this worksheet. There shouldn't need to be any actual code changes beyond the restructure.

8. Once finished, click "Submit and Export"

9. Locally add the generated PDF to a folder of your choosing inside your repository folder and move it to Github

1. `git add .`
2. `git commit -m "adding PDF"`
3. `git push origin Milestone1`
4. On Github merge the pull request from Milestone1 to main

10. Upload the same PDF to Canvas

11. Sync Local

- ## Section #1: (1 pt.) Feature: Server Can Be Started Via Command Line And Listen To Connections

≡ Task #1 (1 pt.) - Evidence

Part 1:

- Show the terminal output of the server started and listening
- Show the relevant snippet of the code that waits for incoming connections

snippet of terminal showing output of the server and listening

code snippet showing for incoming connections

≡ Part 2:

Details:

- Briefly explain how the server-side waits for and accepts/handles connections

Your Response:

the serversocket line of code listens for incoming connections via the port. there is a loop that keeps running awaiting a command . once connected the server logs the connecting then handles it.



Saved: 7/6/2025 10:28:52 AM

Section #2: (1 pt.) Feature: Server Should Be Able To Allow More Than One Client To Be Connected At Once

Progress: 100%

≡ Task #1 (1 pt.) - Evidence

Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the terminal output of the server receiving multiple connections
- Show at least 3 Clients connected (best to use the split terminal feature)
- Show the relevant snippets of code that handle logic for multiple connections

[illegible]

snippet of server receiving multiple connections

[illegible]

continue snippet of server receiving multiple connections

```
OUTPUT: TERMINAL: PORTS: UTILS: DEBUG: CONSOLE
~/Desktop/NOIT/summer/114/IT-114-450 Milestone2-prep
> ./run.sh Project
07/05/2025 20:40:11 [Project.Client.Client] (INFO):
> Client Created
07/05/2025 20:40:11 [Project.Client.Client] (INFO):
> Client starting
07/05/2025 20:40:11 [Project.Client.Client] (INFO):
> Waiting for input
/name Alex1
07/05/2025 20:40:19 [Project.Client.Client] (INFO):
> Name set to Alex1
/connect localhost:3000
07/05/2025 20:40:29 [Project.Client.Client] (INFO):
> Client connected
07/05/2025 20:40:29 [Project.Client.Client] (INFO):
> Connected
07/05/2025 20:40:29 [Project.Client.Client] (INFO):

~/Desktop/NOIT/summer/114/IT-114-450 Milestone2-prep
> ./run.sh Project
07/05/2025 20:40:38 [Project.Client.Client] (INFO):
> Client Created
07/05/2025 20:40:38 [Project.Client.Client] (INFO):
> Client starting
07/05/2025 20:40:38 [Project.Client.Client] (INFO):
> Waiting for input
/name Alex2
07/05/2025 20:40:43 [Project.Client.Client] (INFO):
> Name set to Alex2
/connect localhost:3000
07/05/2025 20:40:52 [Project.Client.Client] (INFO):
> Client connected
07/05/2025 20:40:52 [Project.Client.Client] (INFO):
> Connected
07/05/2025 20:40:52 [Project.Client.Client] (INFO):

~/Desktop/NOIT/summer/114/IT-114-450 Milestone2-prep
> ./run.sh Project
07/05/2025 20:41:01 [Project.Client.Client] (INFO):
> Client Created
07/05/2025 20:41:01 [Project.Client.Client] (INFO):
> Client starting
07/05/2025 20:41:01 [Project.Client.Client] (INFO):
> Waiting for input
/name Alex3
07/05/2025 20:41:05 [Project.Client.Client] (INFO):
> Name set to Alex3
/connect localhost:3000
07/05/2025 20:41:14 [Project.Client.Client] (INFO):
> Client connected
07/05/2025 20:41:14 [Project.Client.Client] (INFO):
> Connected
07/05/2025 20:41:14 [Project.Client.Client] (INFO):
```

snippet of 3 clients successfully connected

```
while (isRunning) {
    info(message:"Waiting for next client");
    Socket incomingClient = serverSocket.accept();
    info(message:"Client connected");

    ServerThread serverThread = new ServerThread(incomingClient, this::onServerThreadInitialized);
    serverThread.start();
}
```

snippet of the code that handles connections

 Saved: 7/5/2025 9:26:54 PM

Part 2:

Progress: 100%

Details:

- Briefly explain how the server-side handles multiple connected clients

Your Response:

the server can handle alot of clients by making new threads for each user that connects. each thread then would communicate with each other without disrupting each other. everything is orgainzed in one place such as the lobby. which is the default place.

 Saved: 7/5/2025 9:26:54 PM

Section #3: (2 pts.) Feature: Server Will Implement The Concept Of Rooms (With The Default Being "l obby")

Progress: 100%

Task #1 (2 pts.) - Evidence

Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the terminal output of rooms being created, joined, and removed (server-side)
- Show the relevant snippets of code that handle room management (create, join, leave, remove) (server-side)

```
> Server: alex1#4 added to lobby
07/06/2025 10:32:00 [Project.Server.ServerThread] (INFO):
> Thread[2]: Received from my client: Payload[ROOM_CREATE] Client Id [0] Message: [AlexRoom]
07/06/2025 10:32:00 [Project.Server.Room] (INFO):
> Room[AlexRoom]: created
07/06/2025 10:32:00 [Project.Server.Server] (INFO):
> Server: created new Room AlexRoom
07/06/2025 10:32:00 [Project.Server.Server] (INFO):
> Server: Removing client from previous Room lobby
07/06/2025 10:32:00 [Project.Server.ServerThread] (INFO):
> Thread[2]: Sending to client: Payload[ROOM_LEAVE] Client Id [4] Message: [null] ClientName: [alex1]
07/06/2025 10:32:00 [Project.Server.ServerThread] (INFO):
> Thread[2]: Sending to client: Payload[MESSAGE] Client Id [4] Message: [Room[lobby] alex1#4 left the room]
07/06/2025 10:32:00 [Project.Server.ServerThread] (INFO):
> Thread[3]: Sending to client: Payload[ROOM_LEAVE] Client Id [4] Message: [null] ClientName: [alex1]
07/06/2025 10:32:00 [Project.Server.ServerThread] (INFO):
> Thread[3]: Sending to client: Payload[MESSAGE] Client Id [4] Message: [Room[lobby] alex1#4 left the room]
07/06/2025 10:32:00 [Project.Server.ServerThread] (INFO):
> Thread[4]: Sending to client: Payload[ROOM_LEAVE] Client Id [4] Message: [null] ClientName: [alex1]
07/06/2025 10:32:00 [Project.Server.ServerThread] (INFO):
> Thread[4]: Sending to client: Payload[MESSAGE] Client Id [4] Message: [Room[lobby] You left the room]
07/06/2025 10:32:00 [Project.Server.ServerThread] (INFO):
> Thread[4]: Sending to client: Payload[ROOM_JOIN] Client Id [-1] Message: [null] ClientName: [null]
07/06/2025 10:32:00 [Project.Server.ServerThread] (INFO):
> Thread[4]: Sending to client: Payload[MESSAGE] Client Id [4] Message: [null] ClientName: [alex1]
07/06/2025 10:32:00 [Project.Server.ServerThread] (INFO):
> Thread[4]: Sending to client: Payload[MESSAGE] Client Id [4] Message: [Room[AlexRoom] You joined the room]
07/06/2025 10:34:23 [Project.Server.ServerThread] (INFO):
> Thread[2]: Received from my client: Payload[ROOM_JOIN] Client Id [0] Message: [AlexRoom]
07/06/2025 10:34:23 [Project.Server.Server] (INFO):
```

snippet on the server-side showing room being created by alex1#4 called Alex1Room and being removed from default (Lobby)

```
> Server: Removing client from previous Room lobby
07/06/2025 10:33:31 [Project.Server.ServerThread] (INFO):
> Thread[2]: Sending to client: Payload[ROOM_LEAVE] Client Id [2] Message: [null] ClientName: [Alex2]
07/06/2025 10:33:31 [Project.Server.ServerThread] (INFO):
> Thread[2]: Sending to client: Payload[MESSAGE] Client Id [2] Message: [Room[lobby] You left the room]
07/06/2025 10:33:31 [Project.Server.ServerThread] (INFO):
> Thread[3]: Sending to client: Payload[ROOM_LEAVE] Client Id [2] Message: [null] ClientName: [Alex2]
07/06/2025 10:33:31 [Project.Server.ServerThread] (INFO):
> Thread[3]: Sending to client: Payload[MESSAGE] Client Id [2] Message: [Room[lobby] Alex2#2 left the room]
07/06/2025 10:33:31 [Project.Server.ServerThread] (INFO):
> Thread[2]: Sending to client: Payload[ROOM_JOIN] Client Id [-1] Message: [null] ClientName: [null]
07/06/2025 10:33:31 [Project.Server.ServerThread] (INFO):
> Thread[2]: Sending to client: Payload[SYNC_CLIENT] Client Id [4] Message: [null] ClientName: [alex1]
07/06/2025 10:33:31 [Project.Server.ServerThread] (INFO):
> Thread[2]: Sending to client: Payload[ROOM_JOIN] Client Id [2] Message: [null] ClientName: [Alex2]
07/06/2025 10:33:31 [Project.Server.ServerThread] (INFO):
> Thread[2]: Sending to client: Payload[MESSAGE] Client Id [2] Message: [Room[AlexRoom] You joined the room]
07/06/2025 10:33:31 [Project.Server.ServerThread] (INFO):
> Thread[4]: Sending to client: Payload[ROOM_JOIN] Client Id [2] Message: [null] ClientName: [Alex2]
07/06/2025 10:33:31 [Project.Server.ServerThread] (INFO):
> Thread[4]: Sending to client: Payload[MESSAGE] Client Id [2] Message: [Room[AlexRoom] Alex2#2 joined the room]
07/06/2025 10:34:23 [Project.Server.ServerThread] (INFO):
> Thread[2]: Received from my client: Payload[ROOM_LEAVE] Client Id [0] Message: [leave]
07/06/2025 10:34:23 [Project.Server.Server] (INFO):
```

snippet of server-side showing room being joined by alex2#2

```
> Server: Removing client from previous Room Alex1Room
07/06/2025 10:34:23 [Project.Server.ServerThread] (INFO):
> Thread[2]: Sending to client: Payload[ROOM_LEAVE] Client Id [2] Message: [null] ClientName: [Alex2]
07/06/2025 10:34:23 [Project.Server.ServerThread] (INFO):
> Thread[2]: Sending to client: Payload[MESSAGE] Client Id [2] Message: [Room[AlexRoom] You left the room]
07/06/2025 10:34:23 [Project.Server.ServerThread] (INFO):
> Thread[4]: Sending to client: Payload[ROOM_LEAVE] Client Id [2] Message: [null] ClientName: [Alex2]
07/06/2025 10:34:23 [Project.Server.ServerThread] (INFO):
> Thread[4]: Sending to client: Payload[MESSAGE] Client Id [2] Message: [Room[AlexRoom] Alex2#2 left the room]
07/06/2025 10:34:23 [Project.Server.ServerThread] (INFO):
> Thread[2]: Sending to client: Payload[ROOM_JOIN] Client Id [-1] Message: [null] ClientName: [null]
07/06/2025 10:34:23 [Project.Server.ServerThread] (INFO):
> Thread[2]: Sending to client: Payload[SYNC_CLIENT] Client Id [3] Message: [null] ClientName: [Alex3]
07/06/2025 10:34:23 [Project.Server.ServerThread] (INFO):
> Thread[2]: Sending to client: Payload[ROOM_JOIN] Client Id [2] Message: [null] ClientName: [Alex2]
07/06/2025 10:34:23 [Project.Server.ServerThread] (INFO):
> Thread[2]: Sending to client: Payload[MESSAGE] Client Id [2] Message: [Room[lobby] You joined the room]
07/06/2025 10:34:23 [Project.Server.ServerThread] (INFO):
> Thread[3]: Sending to client: Payload[ROOM_JOIN] Client Id [2] Message: [null] ClientName: [Alex2]
07/06/2025 10:34:23 [Project.Server.ServerThread] (INFO):
> Thread[3]: Sending to client: Payload[MESSAGE] Client Id [2] Message: [Room[lobby] Alex2#2 joined the room]
```

snippet of server-side showing alex2#2 leaving room Alex1Room

```
break;
case DISCONNECT:
    currentRoom.handleDisconnect(this);
    break;
case MESSAGE:
    currentRoom.handleMessage(this, incoming.getMessage());
    break;
case REVERSE:
    currentRoom.handleReverseText(this, incoming.getMessage());
    break;
```

```

case ROOM_CREATE:
    currentRoom.handleCreateRoom(this, incoming.getMessage());
    break;
case ROOM_JOIN:
    currentRoom.handleJoinRoom(this, incoming.getMessage());
    break;
case ROOM_LEAVE:
    currentRoom.handleJoinRoom(this, Room.LOBBY);
    break;

```

snippet of code showing that handels room management



Saved: 7/6/2025 11:06:29 AM

⇒ Part 2:

Progress: 100%

Details:

- Briefly explain how the server-side handles room creation, joining/leaving, and removal

Your Response:

when the /room command is called, he server calls handleCreateRoom to create a new room. if the room dosnet already existes it is then added. when the /joinroom command is called, the server removes the client from the default room (lobby) and adds them to the room they want to join by using handleJoing room. the same goes for leaving a room. /leave is called and it automacally put u back in the lobby. a room will automatically be removed when they are empty, this is done by the disconnectAll



Saved: 7/6/2025 11:06:29 AM

Section #4: (1 pt.) Feature: Client Can Be Started Via The Command Line

Progress: 100%

≡ Task #1 (1 pt.) - Evidence

Progress: 100%

📄 Part 1:

Progress: 100%

Details:

- Show the terminal output of the /name and /connect commands for each of 3 clients (best to use the split terminal feature)
- Output should show evidence of a successful connection
- Show the relevant snippets of code that handle the processes for /name, /connect, and the confirmation of being fully setup/connected



snippet of 3 different clients added

snippet of connection successful via server

continue snippet of connection successful via server

snippet of /connect code

```
private boolean processClientCommand(String text) throws IOException {
    boolean wasCommand = false;
    if (text.startsWith(Constants.COMMAND_TRIGGER)) {
        text = text.substring(beginIndex+1);

        if (isConnection("/") + text) {
            if (myUser.getClientName() == null || myUser.getClientName().isEmpty()) {
                LoggerUtil.WARNING|
                    TextFX.colorize(text:"Please set your name via /name <name> before connecting", Color.RED));
                return true;
            }
        }
    }
}
```

 Saved: 7/5/2025 9:04:04 PM

≡ Part 2:

Progress: 100%

Details:

- Briefly explain how the /name and /connect commands work and the code flow that leads to a successful connection for the client

Your Response:

the /name command lets the user input there name then the code removes the "/" to check if it is one of the commands avaiable. then it takes the given name and give it to the user. the /name hast to be done in order to continue or else an error would show.

the /connect command alows the user to connect to a server, in this case it would be localhost:3000. then a new socket connection is created to the server this would also allow a place where data can be sent and recived depending on the Number of users.

 Saved: 7/5/2025 9:04:04 PM

Section #5: (2 pts.) Feature: Client Can Create/j oin Rooms

Progress: 100%

≡ Task #1 (2 pts.) - Evidence

Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the terminal output of the /createroom and /joinroom
- Output should show evidence of a successful creation/join in both scenarios
- Show the relevant snippets of code that handle the client-side processes for room creation and joining

```
~/Desktop/NJIT/summer/114/IT-114-450 Milestone2-pr
> ./run.sh Project
07/06/2025 10:31:33 [Project.Client.Client] (INFO):
> Client created
07/06/2025 10:31:33 [Project.Client.Client] (INFO):
> Client starting
07/06/2025 10:31:33 [Project.Client.Client] (INFO):
> Waiting for input
07/06/2025 10:31:38 [Project.Client.Client] (INFO):
> Name set to alex1
> connect localhost:3000
07/06/2025 10:31:44 [Project.Client.Client] (INFO):
> Client connected
07/06/2025 10:31:44 [Project.Client.Client] (INFO):
> Connected
07/06/2025 10:31:44 [Project.Client.Client] (INFO):
> Room1obby1 You joined the room
```

snippet of room Alex1Room being created by alex1#4 ..

snippet showing alex2#2 joining room Alex1room

confirmation that Alex2#2 has joined the room

snippet of code that handles room creation and joining.

the /createroom command allows for user/client to create a room the /join allows for user/client

 Saved: 7/6/2025 12:35:37 PM

Progress: 100%

Progress: 100%

Progress: 100%

[illegible]

case MESSAGE:

```
currentRoom.handleMessage(this, incoming.getMessage());  
break;
```

message being handled in room.



Saved: 7/6/2025 1:18:10 PM

≡ Part 2:

Progress: 100%

Details:

- Briefly explain how the message code flow works

Your Response:

when a message is sent by a client, a payload object with the message is sent through a socket to the server. after the payload is sent it goes to the serverthread where it is recived and make sure its a message. when confirming its a message it is then passed to the "room" the client is in.



Saved: 7/6/2025 1:18:10 PM

Section #7: (1 pt.) Feature: Disconnection

Progress: 100%

≡ Task #1 (1 pt.) - Evidence

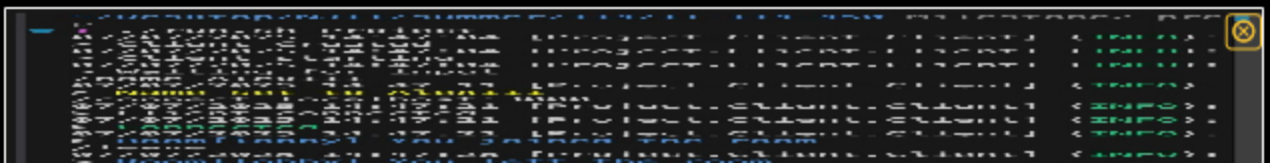
Progress: 100%

📄 Part 1:

Progress: 100%

Details:

- Show examples of clients disconnecting (server should still be active)
- Show examples of server disconnecting (clients should be active but disconnected)
- Show examples of clients reconnecting when a server is brought back online
- Examples should include relevant messages of the actions occuring
- Show the relevant snippets of code that handle the client-side disconnection process
- Show the relevant snippets of code that handle the server-side termination process




```

    } else if (Command.QUIT.command.equalsIgnoreCase(text)) {
        close();
        wasCommand = true;
    } else if (Command.DISCONNECT.command.equalsIgnoreCase(text)) {
        sendDisconnect();
        wasCommand = true;
    }
}

```

snippet of client-side disconnection process code

```

*
* @throws IOException
*/
private void sendDisconnect() throws IOException {
    Payload payload = new Payload();
    payload.setPayloadType(PayloadType.DISCONNECT);
    sendToServer(payload);
}

```

snippet of client-side disconnection process code

```

*/
private void listenToServer() {
    try {
        while (isRunning && isConnected()) {
            Payload fromServer = (Payload) in.readObject(); // blocking read
            if (fromServer != null) {
                processPayload(fromServer);
            } else {
                LoggerUtil.INSTANCE.info(message:"Server disconnected");
                break;
            }
        }
    }
}

```

snippet of client-side disconnection process code

```

private void shutdown() {
    try {
        // chose removeIf over forEach to avoid potential
        // ConcurrentModificationException
        // since empty rooms tell the server to remove themselves
        rooms.values().removeIf(room -> {
            room.disconnectAll();
            return true;
        });
    } catch (Exception e) {
        e.printStackTrace();
    }
}

```

snippet of server-side disconnection process code

```

private Server() {
    Runtime.getRuntime().addShutdownHook(new Thread(() -> {
        info(message:"JVM is shutting down. Perform cleanup tasks.");
        shutdown();
    }));
}

```

snippet of server-side disconnection process code

 Saved: 7/7/2025 3:29:03 PM

 Saved: 7/7/2025 3:29:03 PM

⇒ Part 2:

Progress: 100%

Details:

- Briefly explain how both client and server gracefully handle their disconnect/termination logic

- Details:**

 - Briefly explain how both client and server gracefully handle their disconnect/termination logic

Your Response:

when the client types/ disconnect in the terminal, a DISCONNECT payload is sent to the server. the disconnect method is then called and the socket streams are then closed. for the server the while(is running) is running when the server disconnects the loop is then exited and shutdown is called. after shutdown is called a disconnect method is called and loops through the rooms. finally a shutdown hook ensures clean up

 Saved: 7/7/2025 3:29:03 PM

 Saved: 7/7/2025 3:29:03 PM

Section #8: (1 pt.) Misc

Section #8: (1 pt.) Misc

Task #1 (0.25 pts.) - Show the proper workspace structure with the new Client, Common, Server, and Exceptions packages

Progress: 100%

Task #1 (0.25 pts.) - Show the proper workspace structure with the new Client, Common, Server, and Exceptions packages

Progress: 100%

A screenshot of the IDE's Project Explorer. The 'Project' folder is expanded, showing the 'Client' folder. The 'Client' folder is also expanded, showing the files 'Client.java' and 'Client\$1.class'. The 'Client.java' file is highlighted with a mouse cursor.

A screenshot of the IDE's Project Explorer. The 'Project' folder is expanded, showing the 'Client' folder. The 'Client' folder is also expanded, showing the files 'Client.java' and 'Client\$1.class'. The 'Client.java' file is highlighted with a mouse cursor.

A screenshot of the IDE's Project Explorer. The 'Project' folder is expanded, showing the 'Client' folder. The 'Client' folder is also expanded, showing the files 'Client.java' and 'Client\$1.class'. The 'Client.java' file is highlighted with a mouse cursor.

A screenshot of the IDE's Project Explorer. The 'Project' folder is expanded, showing the 'Client' folder. The 'Client' folder is also expanded, showing the files 'Client.java' and 'Client\$1.class'. The 'Client.java' file is highlighted with a mouse cursor.

A screenshot of the IDE's Project Explorer. The 'Project' folder is expanded, showing the 'Client' folder. The 'Client' folder is also expanded, showing the files 'Client.java' and 'Client\$1.class'. The 'Client.java' file is highlighted with a mouse cursor.

Snippet of client work space

[illegible]

```
RoomNotFoundException.class
RoomNotFoundException.java
RoomNotFoundException.class
RoomNotFoundException.java
RoomNotFoundException.class
RoomNotFoundException.java
RoomNotFoundException.class
RoomNotFoundException.java
```

snippet of common workspace

```
Exceptions
CustomIT114Exception.class
CustomIT114Exception.java
DuplicateRoomException.class
DuplicateRoomException.java
RoomNotFoundException.class
RoomNotFoundException.java
```

snippet of server work space

```
Server
BaseServerThread.class
BaseServerThread.java
BaseServerThread$1.class
Room.class
Room.java
Server.class
Server.java
ServerThread.class
ServerThread.java
ServerThread$1.class
```

snippet of exceptions workspace

```
.gitignore
build.sh
client-0.log
client-0.log.1
client-0.log.1.lock
client-0.log.2
client-0.log.2.lock
client-0.log.3
~/Desktop/N.IT/summer/114/IT-114-450/client-0.log.3
client-0.log.lock
README.md
rm_class_files.sh
run.sh
server-0.log
server-0.log.lock
```

files .sh added.

 Saved: 7/6/2025 1:25:25 PM

Task #2 (0.25 pts.) - Github Details

Progress: 100%

Part 1:

Progress: 100%

Details:

From the Commits tab of the Pull Request screenshot the commit history

Milestone2 prep #6

Open ap2869 wants to merge 2 commits into main from Milestone2-prep

Conversation 0 Commits 2 Checks 0 Files changed 22 +2,432 -1

Commits on Jul 4, 2025

Milestone 1 Baseline
ap2869 committed 2 days ago 8234922

Commits on Jul 5, 2025

Milestone2 Prep Complete
ap2869 committed 16 hours ago 83ba8d7

should have named it mile stone one but it was too late to change. snippet of github commits with link

 Saved: 7/6/2025 11:40:12 AM

Part 2:

Progress: 100%

Details:

Include the link to the Pull Request (should end in `/pull/#`)

URL #1

<https://github.com/ap2869/IT-114-450/pull/6>



URL

<https://github.com/ap2869/IT-114>

 Saved: 7/6/2025 11:40:12 AM

Task #3 (0.25 pts.) - WakaTime - Activity

Progress: 100%

Details:

- Visit the WakaTime.com Dashboard
- Click **Projects** and find your repository
- Capture the overall time at the top that includes the repository name
- Capture the individual time at the bottom that includes the file time
- Note: The duration isn't relevant for the grade and the visual graphs aren't necessary

Projects • IT-114-450

total 27 hrs 30 mins

4 hrs 42 mins over the Last 7 Days in IT-114-450 under all branches.

snippet of waka time on project



snippet of waka time on project



Saved: 7/7/2025 3:31:20 PM

Task #4 (0.25 pts.) - Reflection

Progress: 100%

Task #1 (0.33 pts.) - What did you learn?

Progress: 100%

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

i leared alot from doing milestone one. one big thing that i leared and will possibly continue to learn is to understand errors. u can have a whole prompt of code but if u dont know what needs fixing u basicallt have a piece of metal in front of you. it took alot of going back and fourth different modules and understanding different how different files work with one another.



Saved: 7/6/2025 11:14:48 AM

Task #2 (0.33 pts.) - What was the easiest part of the assignment?

Progress: 100%

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

the easiest part of the assingment was after the set up was finshed and i got

everything to work. after everything worked fine. the commands were stright foward and it was successful and sign of relief.



Saved: 7/6/2025 11:12:13 AM

⇒ Task #3 (0.33 pts.) - What was the hardest part of the assignment?

Progress: 100%

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

the hardest part of the assignment was getting everything to work. had a couple road blocks along the way and had to put in extra hours while doing milestone 1. leared alot on the way. but he set up is usually hard when u dont exactly know what is suppose to happen.but after a while i figured it out.



Saved: 7/6/2025 11:10:17 AM